

WEB TEKNOLOJİLERİ ve PROGRAMLAMA

4. Ders: ReactJS



01 Fragments

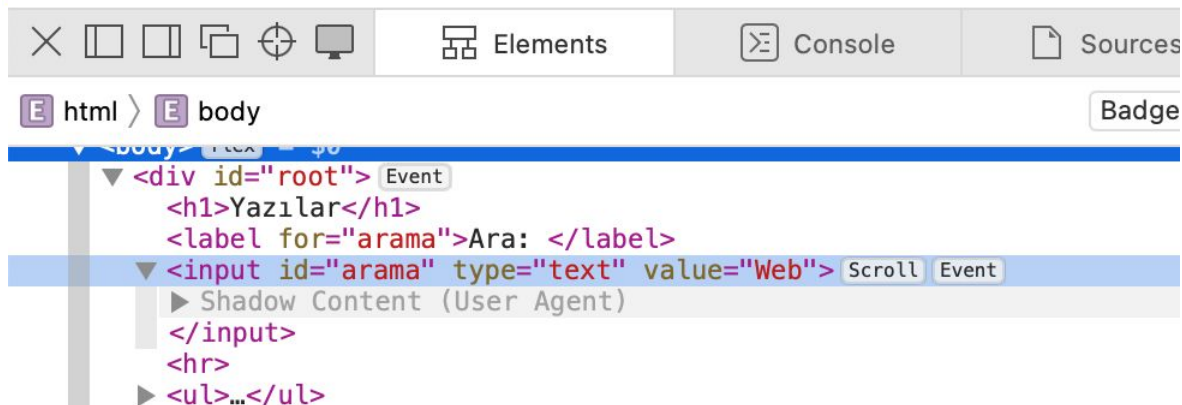
- React bileşenleri genellikle bir kök öğeyle sarmalanmalıdır (div gibi).
- Ancak bazen birden fazla alt bileşeni döndürmek gerekebilir ve bu durumda ek bir gereksiz öge eklemek istenmez.
- Fragments, bu durumu çözmek için kullanılır. Kök bir öge oluşturmadan birden fazla öğeyi gruplamak için kullanılabilir.
- Ana bileşene ekstra bir düzey eklenmez. Böylece, DOM ağacındaki gereksiz düzeylerin oluşmasını önler.
- JSX sözdiziminde `<></>` veya `<React.Fragment></React.Fragment>` etiketleriyle temsil edilir.
- Daha temiz ve düzenli bir JSX kodu yazmayı sağlar. Bileşenin yapısını anlamak ve bakım yapmak kolaylaşır.
- Ana bileşen alt bileşenlerine özellikler geçirebilir ve onlarla etkileşimde bulunabilir. React 16 ve sonraki sürümlerde React Fragments desteklenmektedir.



01 Fragments

- Bu problem çözmek için `<></>` veya `<React.Fragment></React.Fragment>` kullanabilirsiniz.
- Tarayıcınızı açıp öğeyi incele dediğinizde ekstra div elementinin olmadığını göreceksiniz.

```
function Arama({ aramaMetni, onSearch }) {  
  return (  
    <>  
      <label htmlFor="arama">Ara: </label>  
      <input id="arama" type="text" onChange={onSearch} value={aramaMetni} />  
    </>  
  );  
}
```



Örnek: <https://codesandbox.io/s/spring-mountain-lq76gp>

01 Handler Function

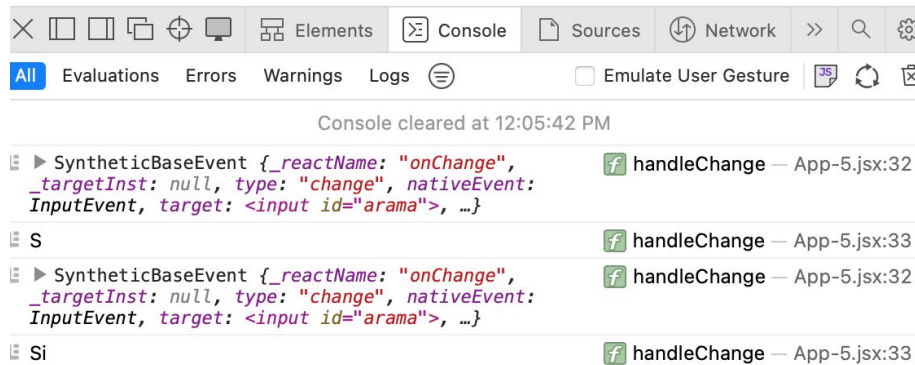
- React bileşenleri oluşturmayı öğrendik. Ancak uygulamamızda etkileşim yok.
- React ile bir uygulama geliştiriyorsanız, bir gün mutlaka bir kullanıcı etkileşimi gerçekleştirmeniz gerekecektir.
- Projemize etkileşim için en uygun yer Arama işlevidir.
- HTML düzeyinde düşündüğümüzde `addEventListener()` metodu aracılığı ile olay yöneticileri ekleyerek etkileşim kurabiliriz.
- React düzeyinde de benzer işlemi yapabiliriz. Arama bileşenimizde olayları yakalayacak bir fonksiyon (event handler) tanımlayabiliriz.
- Bir sonraki aşamada da HTML'in input field elementinin `onChange`(Dikkat edin. JSX camelCase tanımı ile) özelliğine bu fonksiyonu bağlayabiliriz.
- Not: her zaman fonksiyon geçirin, geri dönüş değeri geçirmeyin.
 - `{handleChange()}: Yanlış`
 - `{handleChange}: Doğru`



01 Handler Function

```
function Arama() {  
  const handleChange = (event) => {  
    console.log(event);  
    console.log(event.target.value);  
  };  
  
  return (  
    <div>  
      <label htmlFor="arama">Ara: </label>  
      <input id="arama" type="text" onChange={handleChange} />  
    </div>  
  );  
}
```

Ara:



Örnek: <https://codesandbox.io/s/wonderful-yonath-kfv0kv>

01 Props



- Props ("properties"ın kısaltması) bileşenler arasında veri ve işlevsellik paylaşımını sağlayan bir mekanizmadır.
- Bir bileşen, başka bir bileşene veri veya işlevsellik iletmek için props kullanır.
- Props, bir bileşene dışarıdan geçirilen verileri temsil eder.
- Bir bileşenin props'ları, ebeveyn bileşeni tarafından belirlenir ve çocuk bileşene iletilir.
- Bir bileşenin prop'ları salt okunur ve değiştirilemez nitelikte olmalıdır.
- Props, bir bileşende meydana gelen değişikliklerin genellikle ebeveyn bileşen tarafından yapılması gerektiği anlamına gelir.
- Bir bileşen, aldığı props'ları kullanarak dinamik içerik oluşturabilir veya işlevselliği değiştirebilir.
- Props'lar sadece yukarıdan aşağı doğru iletilebilir. Tersisi mümkün değildir.



01 Props



- `yaziListesi` değişkeni ilk başta App bileşeninde, daha sonra Liste bileşeninde kullandık.
- `yazilListesi` App içinde bir global değişken olarak dursaydı uygulamanız büyüdüğünde, farklı bileşenler farklı klasör ve dosyalarda olduğunda bakım maliyeti ve hata yapma oranınız artacaktı.
- Bunun yerine `yaziListesi`ni bir props'a dönüştürerek istediğimiz bilgiyi ana bileşenden alt bileşenlere geçirebiliriz.
- `yaziListesi`ni props'a dönüştürmeden önce onu App bileşenin bir değişkeni haline getirmemiz lazım. Daha sonra da Liste bileşenine bu değişkeni props olarak geçireceğiz.
 - `Liste(props)`
 - `<Liste yazilar={yaziListesi}/>`



01 Props



```
function App() {  
  const yaziListesi = [  
    {  
      ...  
    }  
  ]  
}
```

```
function Liste(props) {  
  return (  
    <div>  
      <ul>  
        {props.yazilar.map(function (yazi) {  
          return (  
            <li>  
              {yazi.yazisi}</li>  
            )  
          }  
        )}  
      </ul>  
    </div>  
  )  
}
```

```
return (  
  <div>  
    <h1>Yazılar</h1>  
    <Arama />  
    <hr />  
    <Liste yazilar={yaziListesi}/>  
  </div>  
)
```


01 Props

- Artık Liste bileşeni App bileşeninin değişkenine props vasıtasıyla erişebiliyor.
- Peki Liste bileşeninin de Yazi isminde bir alt bileşeni olsaydı yaziListesinin her bir elemanını bu alt bileşene nasıl geçirirdik?

```
function Liste(props) {  
  return (  
    <div>  
      <ul>  
        {props.yazilar.map(function (yazi) {  
          return (  
            <Yazi key={yazi.id} yazi={yazi}/>  
          );  
        })}{" "  
      </ul>  
    </div>  
  );  
}
```

```
function Yazi(props) {  
  return (  
    <li>  
      <span>  
        <a href={props.yazi.url}>{props.yazi.baslik}</a>,  
      </span>  
      <span>  
        <b>Yazar:</b> {props.yazi.yazar}, {" "  
      </span>  
      <span>  
        <b>Yorum Sayısı:</b> {props.yazi.yorum_sayisi}, {" "  
      </span>  
      <span>  
        <b>Puan:</b> {props.yazi.puan}  
      </span>  
    </li>  
  );  
}
```

Örnek: <https://codesandbox.io/s/reverent-tu-jxt3r2>

01 State



- Props'da verileri değiştiremiyorduk. Sadece bir ana bileşenden alt bileşene geçirebiliyorduk. State bu noktada devreye girer.
- State, bir bileşenin içindeki verilerin tutulduğu bir JavaScript nesnesidir. Bu veriler bileşene özgüdür ve bileşenin durumunu temsil eder. Değiştirilebilir niteliktedir.
- State, bileşenin yaşam döngüsü boyunca güncellenebilir. Bileşenin başlangıç durumu tanımlanır ve daha sonra kullanıcı etkileşimi veya veri değişiklikleri gibi olaylara yanıt olarak güncellenebilir.
- State, sadece içinde bulunduğu bileşene özgüdür ve başka bileşenlerle paylaşılmaz. Veri iletişimi için props kullanılır.
- State, bileşenin "setState" veya hook fonksiyonu "useState" kullanılarak güncellenebilir. State güncellendiğinde, React otomatik olarak bileşenin yeniden render edilmesini sağlar, böylece güncel veriler görüntülenir.



01 State



- State kavramını bir örnek yaparak anlamaya çalışalım. Kullanıcının Arama bileşeninde yer alan arama kutusuna yazdığı metni anlık olarak "<metin> aranıyor..." şeklinde ekrana yazacak bir özellik ekleyelim.
- State kullanabilmemiz için önce react'i import etmemiz lazım.
 - `import React from 'react'`
- useState hook metodu aracılığı ile aramaMetni isminde bir State oluşturmamız. useState içindeki ifade ilk durumu temsil eder.
 - `const [aramaMetni, setAramaMetni] = React.useState("");`
- Son olarak da handleChange metodunda ekrandaki her bir değişiklikte gelen veriyi state metoduna göndermeliyiz.
 - `setAramaMetni(event.target.value);`
- useState metodunu kullanarak React'a durumu sürekli değişen bir veri olduğunu bildiriyoruz. Anlık durum aramaMetni değişkeninde tutuluyor. setAramaMetni metodu ise durum her değiştiğinde aramaMetni değişkenini yeni veriyle dolduruyor.



01 State

```
function Arama() {  
  const [aramaMetni, setAramaMetni] = React.useState("");  
  const handleChange = (event) => {  
    setAramaMetni(event.target.value);  
  };  
  
  return (  
    <div>  
      <label htmlFor="arama">Ara: </label>  
      <input id="arama" type="text" onChange={handleChange} />  
      <p>  
        <strong>{aramaMetni} aranıyor...</strong>.  
      </p>  
    </div>  
  );  
}
```

Yazılar

Ara:

Sinan aranıyor....

- [React Öğreniyorum](#), Yazar: Sinan Yüksel, Yorum Sayısı: 3, Puan: 4
- [Web Teknolojileri ve Programlama](#), Yazar: Asım Yüksel, Yorum Sayısı: 2, Puan: 5

Örnek: <https://codesandbox.io/s/state-qyh8yi>

01 State

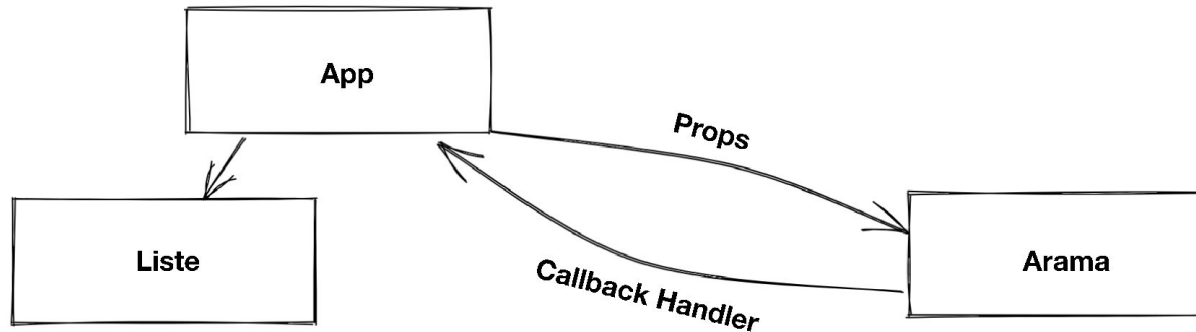


- Kullanıcı giriş alanına yazdığında, giriş alanının change olayı, event handler metoduna girer. Handler, güncellenen state'i ayarlamak için olayın hedef değerini ve state güncelleyici metodu kullanır. Daha sonra bileşen yeniden render edilir. Güncellenen state, geçerli state haline gelir (aramaMetni) ve bileşenin JSX'inde görüntülenir.
- Bir React uygulamasındaki her bileşenin bir ilk render ve ardından potansiyel re-render işlemleri vardır. Genellikle ilk render, tarayıcıda bir React bileşeni görüntülendiğinde gerçekleşir. Ardından, bir kullanıcı etkileşimi gibi bir side-effect meydana geldiğinde (örneğin, bir giriş alanına yazmak), değişiklik, bu değişiklikten etkilenen tüm bileşenlerin yeniden oluşturulmasını zorlayan React'in state'inde yakalanır.
- Arayüz ilk defa oluşturulduğunda/render useState hook ilk değeriyle başlatılır. State değişikliğinden dolayı arayüzün her yeniden oluşturulmasında useState en son değeri kullanır.



01 Callback Handler

- Normalde veriler, bileşen hiyerarşisinde yukarıdan aşağıya, yani üst bileşenden alt bileşene (parent to child) props aracılığıyla aktarılır.
- Ancak, bir alt bileşenden üst bileşene veri aktarmak gerektiğinde, callback fonksiyonları etkili bir çözüm sağlar.
- Bu yöntemde, üst bileşen bir fonksiyonu (callback) alt bileşene prop olarak geçirir.
- Alt bileşen, bir olay meydana geldiğinde (örneğin bir formun gönderilmesi veya bir butona tıklanması), bu callback fonksiyonunu çağırır ve veriyi üst bileşene aktarır.



01 Callback Handler

```
//1. aşama: callback metodunun oluşturulması
const handleSearch = (event) => {
  console.log(event.target.value);
};

//2. aşama: callback metodunun
// Arama bileşenine onSearch props olarak geçirilmesi
return (
  <div>
    <h1>Yazılar</h1>
    <Arama onSearch={handleSearch} />
  </div>
);

//3. aşama: Arama bileşenine props eklenmesi
function Arama(props) {
  const [aramaMetni, setAramaMetni] = React.useState('');
  const handleChange = (event) => {
    setAramaMetni(event.target.value);
  };
  //4. aşama: props aracılığı ile
  //App bileşeninin callback metodunun çağırılması.
  props.onSearch(event);
};
```

Örnek: <https://codesandbox.io/s/callback-handler-r94ggz>

01 Lifting State



- React bileşenleri, bağımsız ve tekrar kullanılabilir parçacıklardır. Her bir bileşen, kendine ait bir durum(state) yönetebilir ve render edebilir.
- Ancak bazen, birden fazla bileşen aynı veriyi paylaşmalı veya aynı veriyi güncellemelidir. Bu durumda, bu veri yukarı yönlü olarak daha üst bir bileşende yönetilebilir.
- Lifting state işlemi, bu ortak veriyi üst bileşene taşımak anlamına gelir. Bu, paylaşılan verinin tek bir kaynaktan yönetilmesini ve tutarlılığını sağlar.
- Paylaşılan veri, üst bileşenin durumu(state) olarak tutulur ve bu veri alt bileşenlere "props" olarak aktarılır.
- Alt bileşenler, aldıkları veriyi kullanarak işlemlerini gerçekleştirir veya görünümelerini günceller. Ancak, bu veriyi doğrudan değiştiremezler, sadece değişiklikleri üst bileşene iletebilirler.
- Değişiklikler, üst bileşende gerçekleştirilir ve güncellenen veri tekrar alt bileşenlere "props" olarak aktarılır.



01 Lifting State



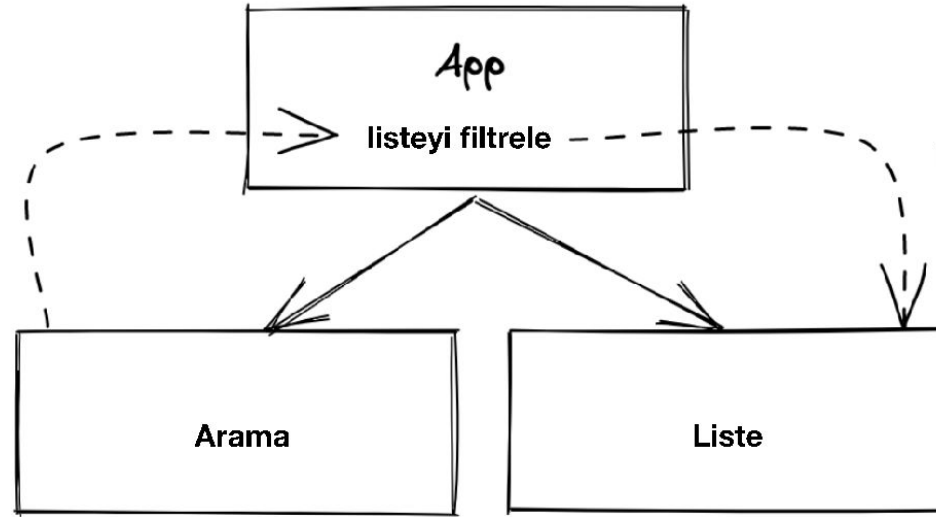
- Kendi uygulamamızda arama işlemine yeni bir özellik ekleyelim. Ne zaman bir metin girilse girilen metne göre yazıListesinde başlık ile eşleşen verileri filtreleyip ekranda göstermeye çalışalım.
- Şu anda aramaMetni değişkeni Arama bileşeninde bir durum olarak tutuluyor. App bileşeninin buna doğrudan müdahale etme şansı yok.
- **İlk akla gelen yöntem:** App bileşeninde Arama bileşeninden gelen aramaMetnini tutatacak başka bir state tanımlamak, filtre işlemini uygulamak ve filtreli veriyi tekrar Arama bileşenine props olarak geçirmek. Bu yöntem kod tekrarına yol açacaktır ve önerilmez.
- **En iyi yöntem:** aramaMetni'ni App bileşenine taşımaktır. Daha sonra state güncelleyen setAramaMetni fonksiyonunu callback handler olarak Arama bileşenine geçirmeliyiz. Son olarak App bileşeni state güncellemelerini aldığı anda yazıListesini filtreleyerek ekranda göstermeli.



01 Lifting State



callback handler ile
aramaMetni state'ini App
bileşeninde güncelle



Filtrelenmiş listeyi props ile
Liste bileşenine gönder.

Kurallar:

1. Eğer bir alt bileşenin state'i güncellemesi gerekiyorsa o bileşene callback handler geçirin.
2. Eğer bir bileşenin state'i kullanması gerekiyorsa state'i o bileşene props olarak geçirin.

01 Lifting State

- Filtreleme işlemini array veri yapısının filter metodu ile yapabiliriz.
- Bu metot yaziListesi üzerinde başlığa göre filtreleme yapıp arananYazilar isimli bir diziye filtreli verileri aktaracak. Liste bileşeni artık yaziListesi yerine arananYazilar dizisini kullanacak.

```
function App() {  
  const [aramaMetni, setAramaMetni] = React.useState("");
```

```
  const arananYazilar = yaziListesi.filter(function (yazi) {  
    return yazi.baslik.toLowerCase().includes(aramaMetni.toLowerCase());  
  });
```

```
  <Arama onSearch={handleSearch} />  
  <hr />  
  <Liste yazilar={arananYazilar} />
```

```
function Arama(props) {  
  return (  
    <div>  
      <label htmlFor="arama"> Arama:</label>  
      <input id="arama" type="text" onChange={handleChange} />  
    </div>  
  );  
}
```

Örnek: <https://codesandbox.io/s/vibrant-raman-s4gqvd>

01 Controlled Components

- App bileşeninde aramaMetni state'ini boş string yerine 'React' ifadesiyle başlatsak ne gibi bir problem olurdu?
 - `React.useState("React");`

Ara:

- [React Öğreniyorum](#), Yazar: Sinan Yüksel, Yorum Sayısı: 3, Puan: 4

- Yukarıdaki görselde gördüğünüz gibi arama kutusu boş ancak state'i 'React' ile başlattığımız için filtreme gerçekleşti.
- Bu problemi çözmek için Arama bileşenini kontrollü bileşen haline getirmemiz gerekir. Bu şekilde HTML'in input field elementini de kontrol edebiliriz ve **value** özelliğine ilk değer ataması yapabiliriz.



01

Controlled Components



```
function App() {  
  const [aramaMetni, setAramaMetni] = React.useState("React");  
  
  <h1>Yazılar</h1>  
  <Arama aramaMetni={aramaMetni} onSearch={handleSearch} />  
  <hr />  
  
  <label htmlFor="arama">Ara: </label>  
  <input id="arama" type="text" onChange={props.onSearch} value={props.aramaMetni}/>  
}
```

Ara:

- [React Öğreniyorum](#), Yazar: Sinan Yüksel, Yorum Sayısı: 3, Puan: 4

Örnek: <https://codesandbox.io/s/practical-nobel-mncxc0>

01 Props Handling

- Props'lar Javascript nesneleridir. Bileşenlere props geçirirken daha anlaşılır değişken isimleri kullanabiliriz. Bu noktada Javascript'in **object destructuring** yöntemini kullanabiliriz.
- Bu yöntemi kullanmamış olsak aşağıdaki gibi değişken ataması yapacaktık.

```
const kullanıcı = { ad: "Sinan", soyad: "Yüksel" };  
  
const ad = kullanıcı.ad;  
const soyad = kullanıcı.soyad;
```

- Bu yöntemi kullanarak artık aşağıdaki gibi bir atama yapabiliriz.

```
const kullanıcı = { ad: "Sinan", soyad: "Yüksel" };  
const { ad, soyad } = kullanıcı;
```

- Şimdi aynı işlemi props için yapalım. Ek olarak da spread operatörünü kullanalım. Örnek: {...yazi}



01 Props Handling



```
function Arama({aramaMetni,onSearch}) {  
  return (  
    <div>  
      <label htmlFor="arama">Ara: </label>  
      <input id="arama" type="text" onChange={onSearch} value={aramaMetni}/>  
    </div>  
  );  
}
```

```
function Liste({yazilar}) {  
  return (  
    <div>  
      <ul>  
        {yazilar.map(function (yazi) {  
          return <Yazi key={yazi.id} {...yazi} />;  
        })}      </ul>  
    </div>  
  );  
}
```

```
function Yazi({url,baslik,yazar,yorum_sayisi,puan}) {  
  return (  
    <li>  
      <span>  
        <a href={url}>{baslik}</a>,  
      </span>  
      <span>  
        <b>Yazar:</b> {yazar},{ " "  
      </span>  
      <span>  
        <b>Yorum Sayısı:</b> {yorum_sayisi},{ " "  
      </span>  
      <span>  
        <b>Puan:</b> {puan}  
      </span>  
    </li>  
  );  
}
```

Örnek: <https://codesandbox.io/s/props-handling-u8stpt>

01 Side-Effects

- React bileşenleri, render işlemi sırasında arayüzünü oluşturmak için JSX kullanır.
- Ancak bazen bileşenlerin yalnızca arayüz oluşturmakla kalmayıp, belirli durumlarda ekstra işlevleri de yerine getirmesi gerekebilir.
- Side effects, bu ekstra işlevleri gerçekleştirmek için kullanılır.
- Örneğin, API çağrıları yapmak, veri tabanı güncellemeleri yapmak, olay dinlemek, animasyonlar gerçekleştirmek gibi işlemler side effects olarak kabul edilir.
- React bileşenlerinde side effects için genellikle useEffect adlı bir "hook" kullanılır. useEffect, bileşenin yaşam döngüsüne bağlı olarak çalışan ve side effects'i gerçekleştiren bir fonksiyonu içine alır.
- useEffect hook'u, bileşenin mount (ilk render), güncellenmesi (props veya state değişiklikleri) ve unmount (bileşen kaldırıldığında) gibi belirli durumlarda çalışır.



01 Side-Effects



- Side effect fonksiyonu, genellikle birinci parametre olarak geçilen bağımlılık dizisi tarafından tetiklenir.
- Bu dizideki değerler, side effect fonksiyonunun çalışmasını tetikleyen bileşenin props veya state değerleridir.
- Bağımlılık dizisi boş bırakılırsa, side effect fonksiyonu sadece bileşenin ilk renderinde çalışır.
- Side effect fonksiyonu içinde yapılan işlemler, genellikle etkileşimli veya asenkron işlemlerdir.
- Örneğin, API çağrıları yapmak için fetch fonksiyonu kullanılabilir veya bir abonelik oluşturmak için olay dinleyicileri eklenir.
- Kendi uygulamamızda da side effect kullanarak aranan metnin hatırlanmasını sağlayacak bir yapı kuralım. Kullanıcı tarayıcıyı kapattığında son aranan metni hatırlasın ve tarayıcı tekrar açıldığında bu metin arama kutucuğuna tekrar eklensin.



01 Side-Effects



- Hatırlama işlemi için localStorage nesnesini kullanacağız.
 - `localStorage.setItem('aranan', event.target.value);`
- Uygulamayı açtığımızda eğer localStorage boş ise 'React' anahtar kelimesini arama kutusuna ekleyeceğiz.
- Eğer daha önce arama yapılmışsa ve localStorage'e kaydedilmişse localStorage'da tutulan arama metnini arama kutusuna ekleyeceğiz.
 - `const [aramaMetni, setAramaMetni] = React.useState(localStorage.getItem('aranan') || 'React');`
- `React.useEffect`'de ise side effect şartını `arananMetin`'in değişmesi şartına bağlayacağız.
 - `[aramaMetni]`
- Uygulamayı açıp arama yaptıktan sonra tarayıcıyı kapatıp yeniden açtığınızda son arama ekranda görünecektir.



01 Side-Effects



```
function App() {  
  const [aramaMetni, setAramaMetni] = React.useState(  
    localStorage.getItem("aranan") || "React"  
  );  
  
  const handleSearch = (event) => {  
    setAramaMetni(event.target.value);  
    localStorage.setItem("aranan", event.target.value);  
  };  
  
  React.useEffect(() => {  
    localStorage.setItem("aranan", aramaMetni);  
  }, [aramaMetni]);  
}
```

Örnek: <https://codesandbox.io/s/side-effects-l6t0x7>

01 Ödev-2 (5 puan)

- Okul kulübünün küçük bir web kitaplığı var. Kullanıcılar bir liste içindeki kitapları **başlığa** göre arayacak, **kategoriye** göre filtreleyecek ve bazılarını **favorilere ekleyip** sonra geri dönebilecek. Uygulama kapatılıp açıldığında **son arama metni** ve **favoriler** hatırlanmalı.
- Bileşenler**
 - App:** kitaplar (sabit dizi), aramaMetni, kategori, favoriler state'leri; filtreleme burada yapılacak.
 - AramaCubugu:** input kontrollü bileşen; value={aramaMetni}
 - KategoriFiltre:** select kontrollü bileşen; seçimi setKategori ile App'e bildirir
 - KitapListe:** Filtrelenmiş diziyi prop olarak alır ve map'ler.
 - KitapKartı:** { baslik, yazar, kategori, id, favorideMi } gibi props'ları **destructuring** ile alır; "Favori / Çıkar" butonu üst bileşene bildirir.
 - FavoriPaneli:** Favori sayısını ve listesini gösterir.
- İlk ödevde yaptığınız gibi bir repo oluşturuyorsunuz. Kodunuzu bu repoya gönderiyorsunuz.
- Readme.md dosyası hazırlayarak uygulamanızın vercel linkini readme dosyanıza ekliyorsunuz. Aşağıdaki formu doldurarak ödevinizi gönderiyorsunuz.
- Ödev Gönderim Formu



Mini Kitaplık

Tümü

React'e Giriş

D. Usta · Web

★ Favoride

İleri JavaScript

S. Kılıç · Web

★ Favoride

Veri Yapıları

A. Demir · CS

☆ Favori Ekle

Algoritmalar

E. Kaya · CS

☆ Favori Ekle

UI/UX Temelleri

N. Akın · Tasarım

☆ Favori Ekle

Favoriler (2)

- React'e Giriş Kaldır
- İleri JavaScript Kaldır